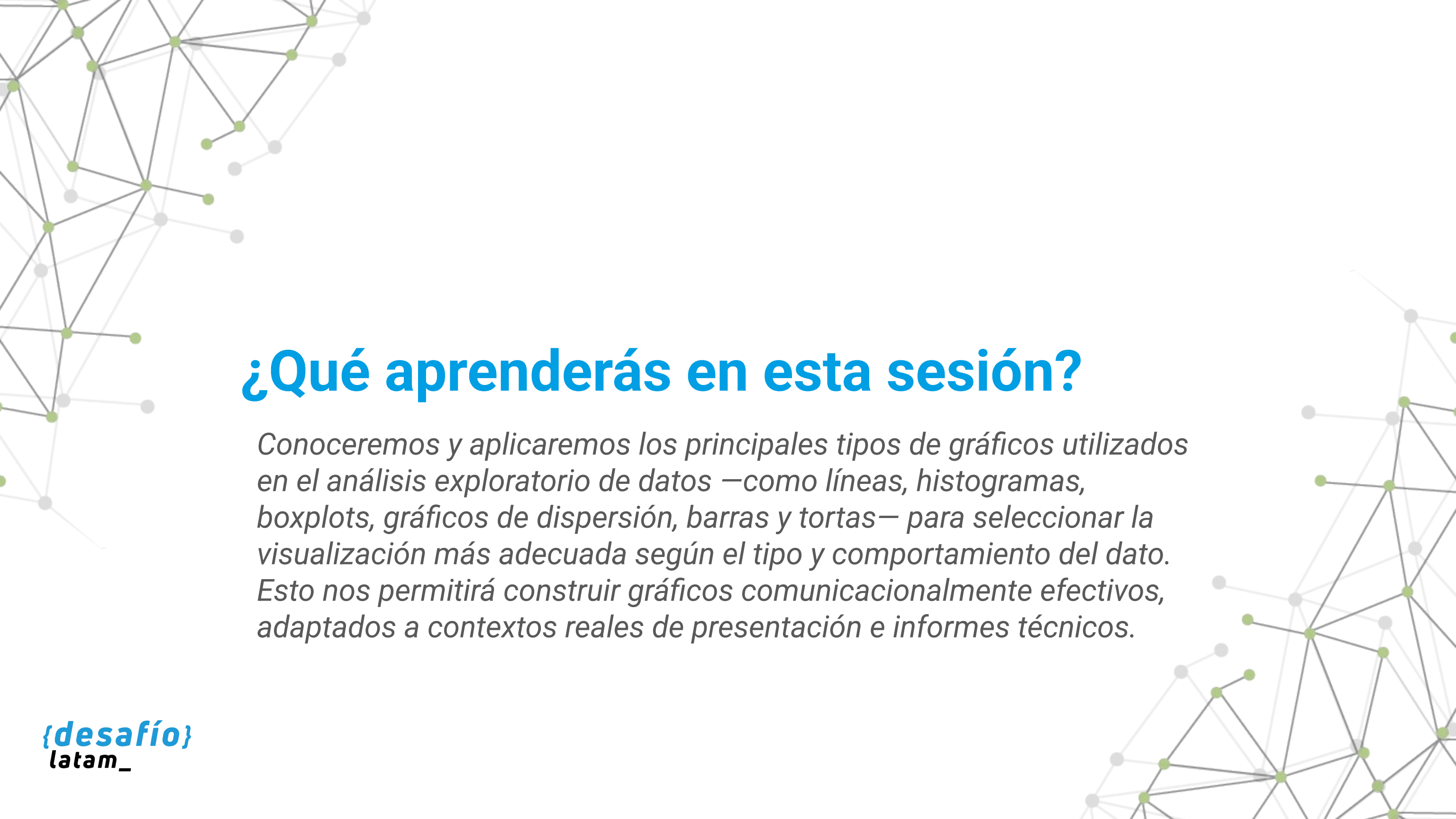




Visualización de datos

● Clase 8 - Tipos de gráficos y herramientas para la comunicación visual



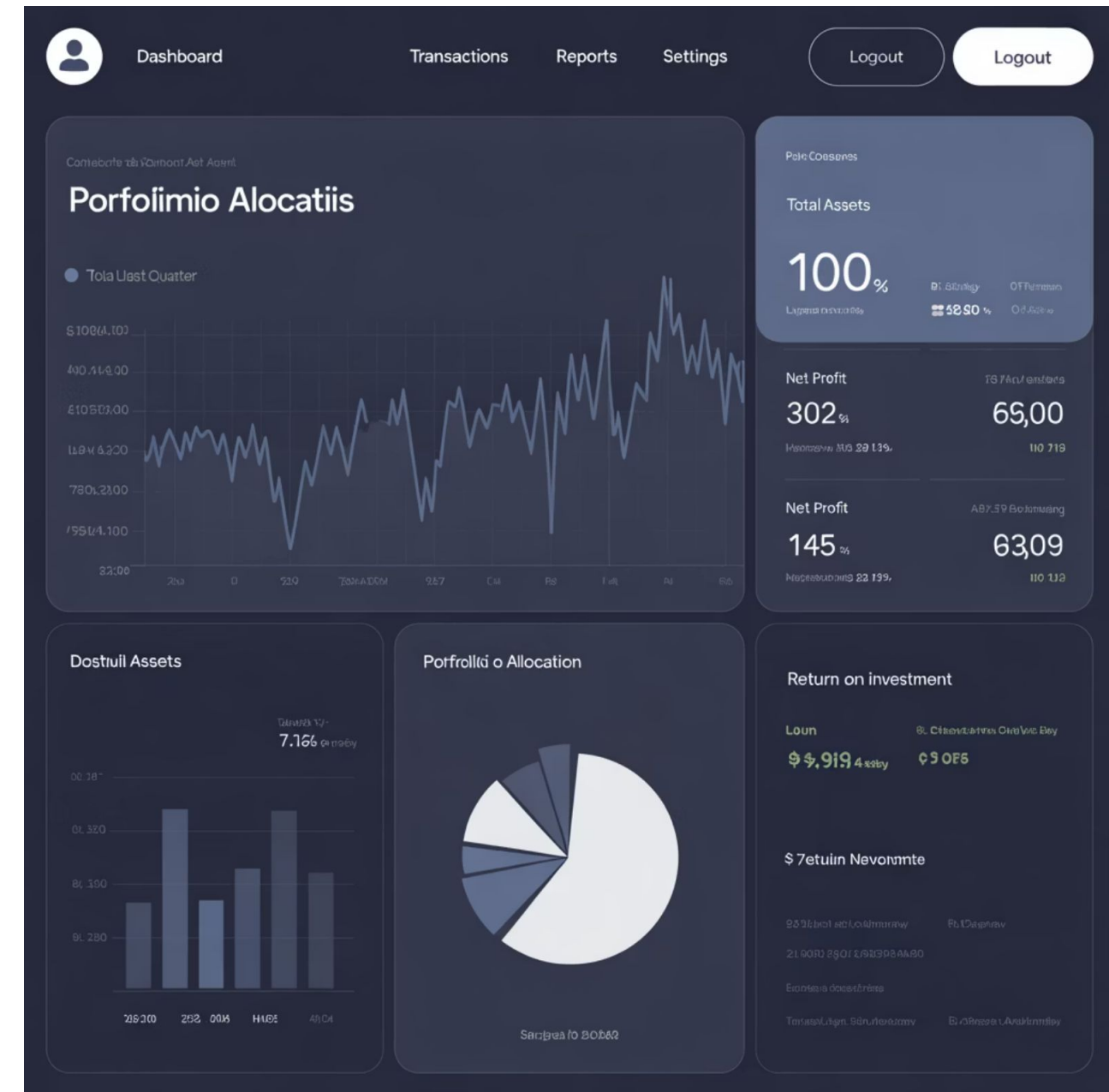
¿Qué aprenderás en esta sesión?

Conoceremos y aplicaremos los principales tipos de gráficos utilizados en el análisis exploratorio de datos –como líneas, histogramas, boxplots, gráficos de dispersión, barras y tortas– para seleccionar la visualización más adecuada según el tipo y comportamiento del dato. Esto nos permitirá construir gráficos comunicacionalmente efectivos, adaptados a contextos reales de presentación e informes técnicos.

Desafío inicial

Estás preparando un informe de análisis de datos para una empresa del sector financiero. Debes comunicar hallazgos clave sobre tendencias temporales, dispersión de ingresos por segmento, distribución de edades, proporción de tipos de cuenta, y detectar valores atípicos. El equipo ha solicitado no solo visualizaciones claras y funcionales, sino también exportables, con detalles resaltados y límites definidos que no distorsionen la interpretación.

¿Cómo construir una colección de gráficos variados y adecuados que comuniquen información de forma precisa, profesional y visualmente clara, incluyendo límites personalizados, anotaciones explicativas y opciones de exportación?



/* Fijando los límites de un gráfico */

Fijando los límites de un gráfico

¿Qué efecto puede tener dejar que un gráfico defina automáticamente los límites de sus ejes?

En análisis de datos, la escala y el rango de los ejes de un gráfico influyen directamente en la percepción de las tendencias, variaciones y comparaciones. Matplotlib define automáticamente los límites según los datos disponibles, pero en muchos casos esto puede conducir a interpretaciones erróneas o exageradas.

Por ejemplo, si estamos mostrando un cambio de temperatura de 25.2 °C a 25.6 °C, y el eje Y va solo de 25.1 a 25.7, el gráfico parecerá mostrar una variación drástica, cuando en realidad es mínima. Es por ello que definir manualmente los límites resulta clave para una comunicación honesta y profesional.

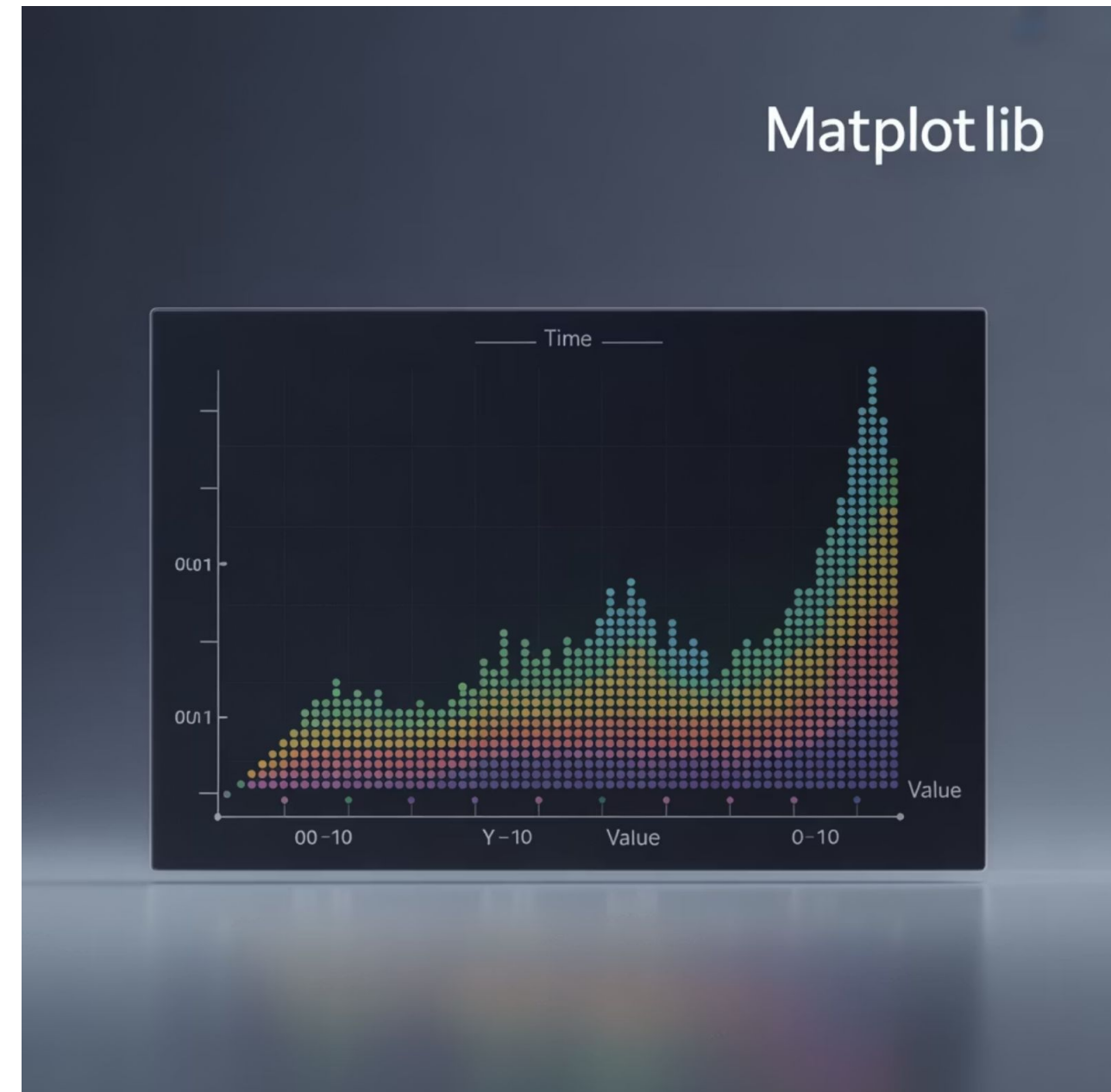


Establecer límites con set_xlim() y set_ylim()

Podemos definir los límites del eje X o Y de forma explícita:

```
import matplotlib.pyplot as plt
x = [1, 2, 3, 4, 5]
y = [20, 22, 21, 23, 24]
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlim(0, 6) # Límite del eje X de 0 a 6
ax.set_ylim(18, 26) # Límite del eje Y de 18 a 26
ax.set_title('Temperatura semanal')
plt.show()
```

Esto asegura que el gráfico se mantenga dentro de rangos razonables y facilita la comparación entre múltiples visualizaciones que usen la misma escala.



Ajuste automático y sus riesgos

Matplotlib también permite límites automáticos mediante `autoscale()`:

```
ax.autoscale(enable=True, axis='both', tight=False)
```

Aunque útil para gráficos exploratorios, el uso exclusivo de escalas automáticas puede generar:

- Exageración de variaciones pequeñas
- Pérdida de consistencia entre gráficos comparativos
- Ocultamiento de valores atípicos si estos definen todo el rango

Comparación visual: escalas automáticas vs. manuales

Configuración	Ventaja principal	Riesgo principal
<code>autoscale()</code>	Rápido para gráficos únicos	Escalas inconsistentes o exageradas
<code>set_xlim()</code> / <code>set_ylim()</code>	Control total del rango	Requiere conocimiento previo del dato

Buenas prácticas para controlar límites

Controlar los límites permite representar de forma más honesta y clara la información. Algunas buenas prácticas clave:

Unificar rangos en gráficos comparativos

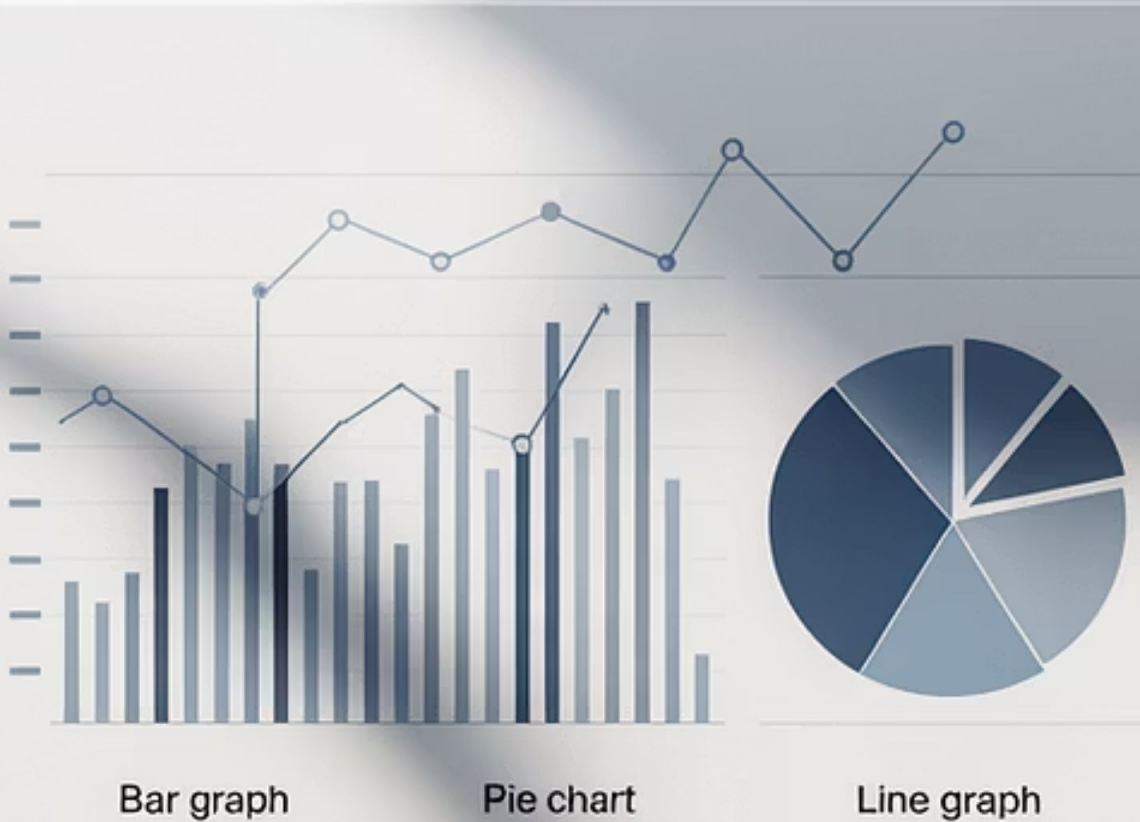
Permite detectar diferencias reales entre distintos conjuntos de datos.

Evitar zooms engañosos

Comunicar explícitamente el rango visualizado para no distorsionar la percepción.

Mostrar outliers sin distorsión

Considerar ejes con escalas logarítmicas si es necesario para mantener la integridad visual.



Errores comunes y cómo evitarlos

Error	Causa probable	Solución recomendada
Gráfico exagera diferencias	Rango del eje muy ajustado	Ampliar con <code>set_ylim()</code>
Comparación inconsistente	Escalas distintas entre gráficos	Usar mismos límites manuales
Elementos fuera de vista	Límites muy estrechos	Aplicar <code>autoscale()</code> o ampliar manualmente

Definir los límites de los ejes es una práctica esencial para asegurar que los datos se interpreten correctamente.

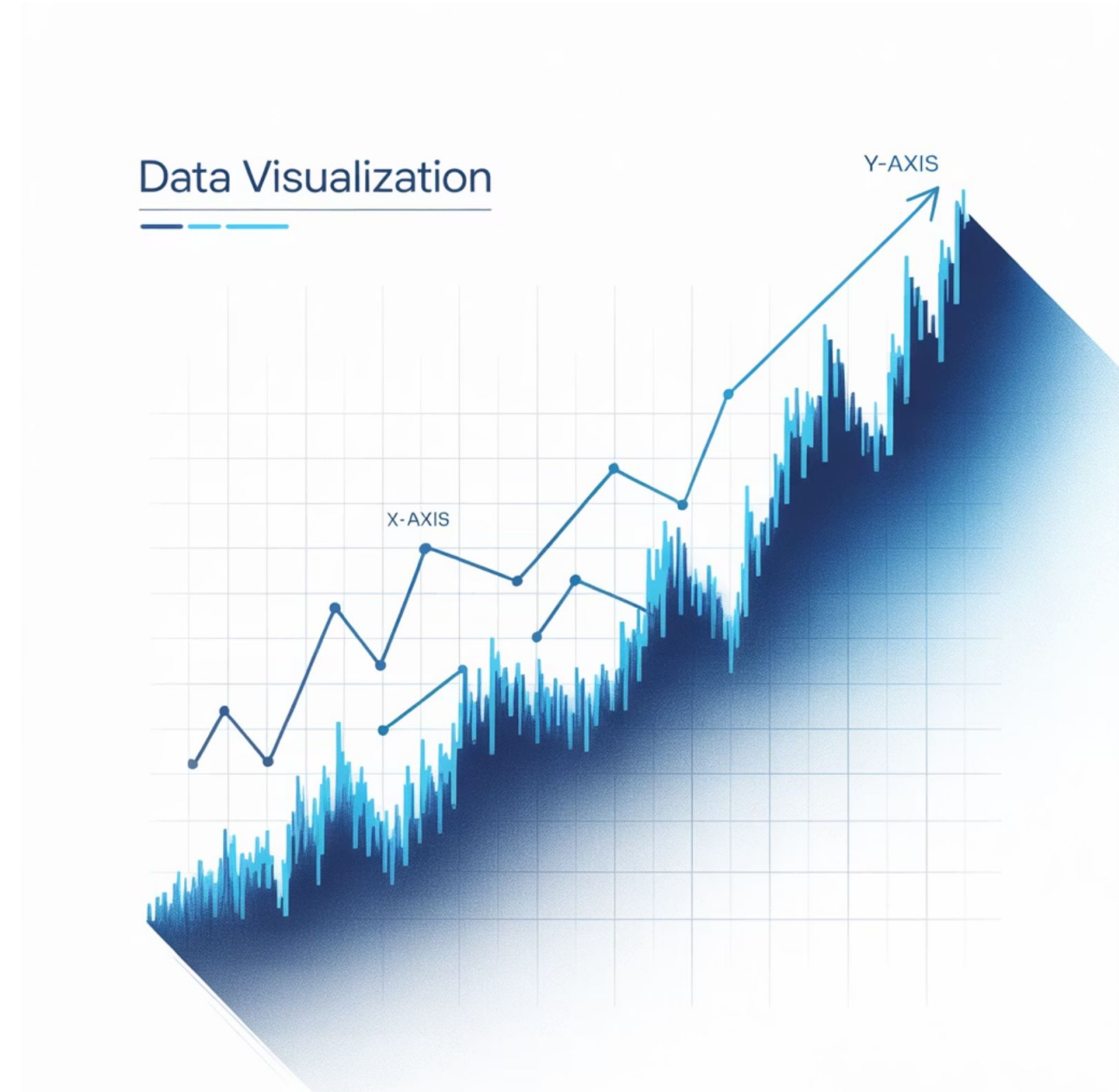
Ahora que comprendemos cómo controlar el rango visualizado, avanzaremos hacia la personalización de los gráficos con elementos adicionales como anotaciones, flechas y marcas sobre la visualización.

Anotaciones y dibujos en un gráfico

¿Cómo podemos destacar valores clave o eventos importantes dentro de una visualización de forma clara y profesional?

En el análisis de datos, no basta con mostrar tendencias generales; a menudo, es necesario llamar la atención sobre ciertos puntos, cambios o eventos específicos que tienen relevancia analítica. Las anotaciones permiten agregar comentarios, etiquetas o marcas sobre el gráfico, mejorando la comunicación visual sin sobrecargar la gráfica de información externa.

Matplotlib ofrece herramientas como `annotate()`, `text()` y elementos gráficos como líneas (`axhline`, `axvline`) y formas (`patches`) para enriquecer los gráficos, resaltar anomalías o guiar la interpretación.

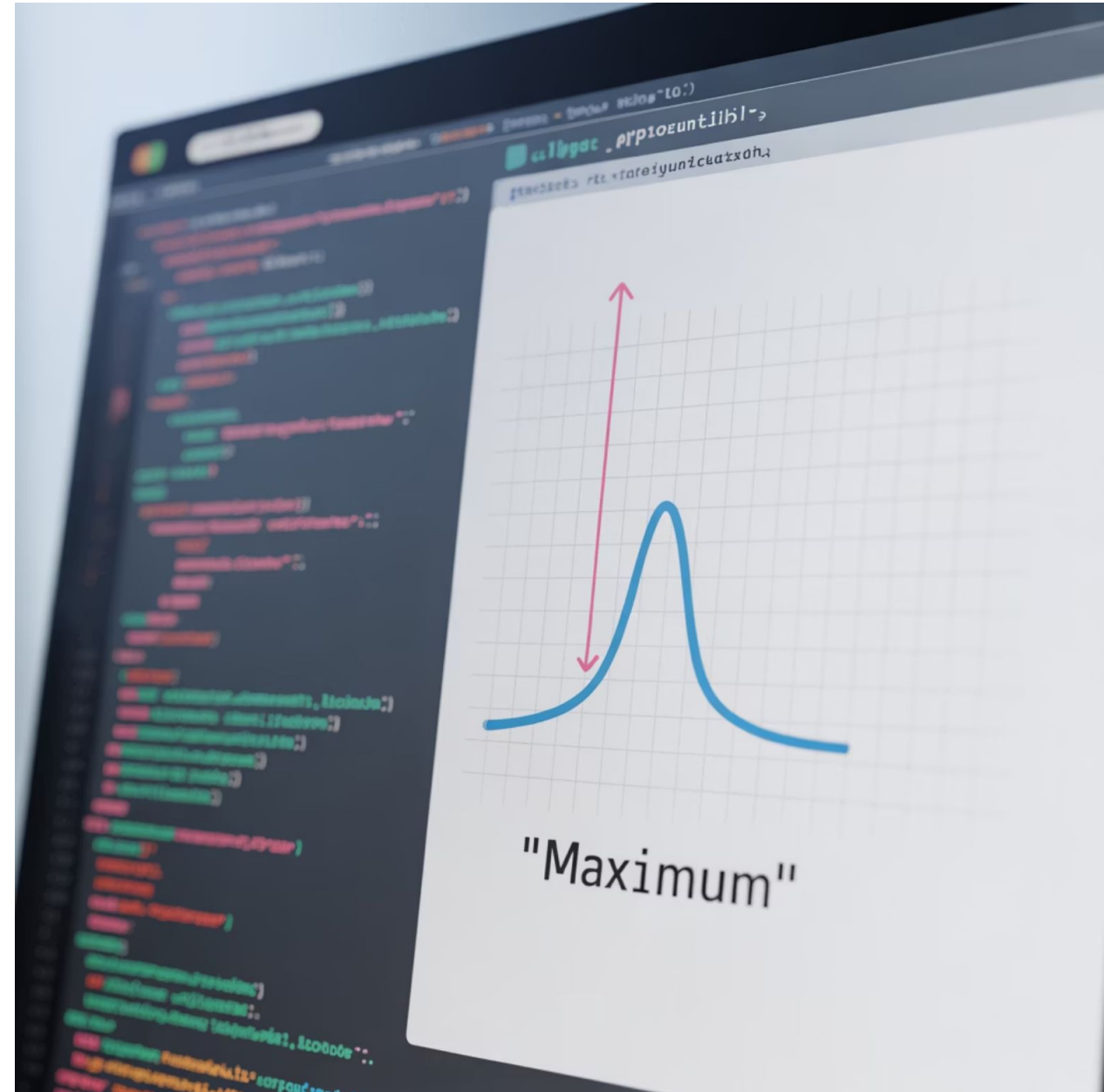


Añadir texto con annotate()

La función `annotate()` permite señalar un punto específico y acompañarlo con una etiqueta y una flecha:

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3, 4], [10, 20, 15, 25])
ax.annotate('Pico máximo', xy=(4, 25),
            xytext=(3, 26),
            arrowprops=dict(facecolor='red',
                            arrowstyle='->'))
ax.set_title('Anotación destacada')
plt.show()
```

Esta función es altamente personalizable: se pueden modificar el color, grosor, estilo de flecha y posición del texto.

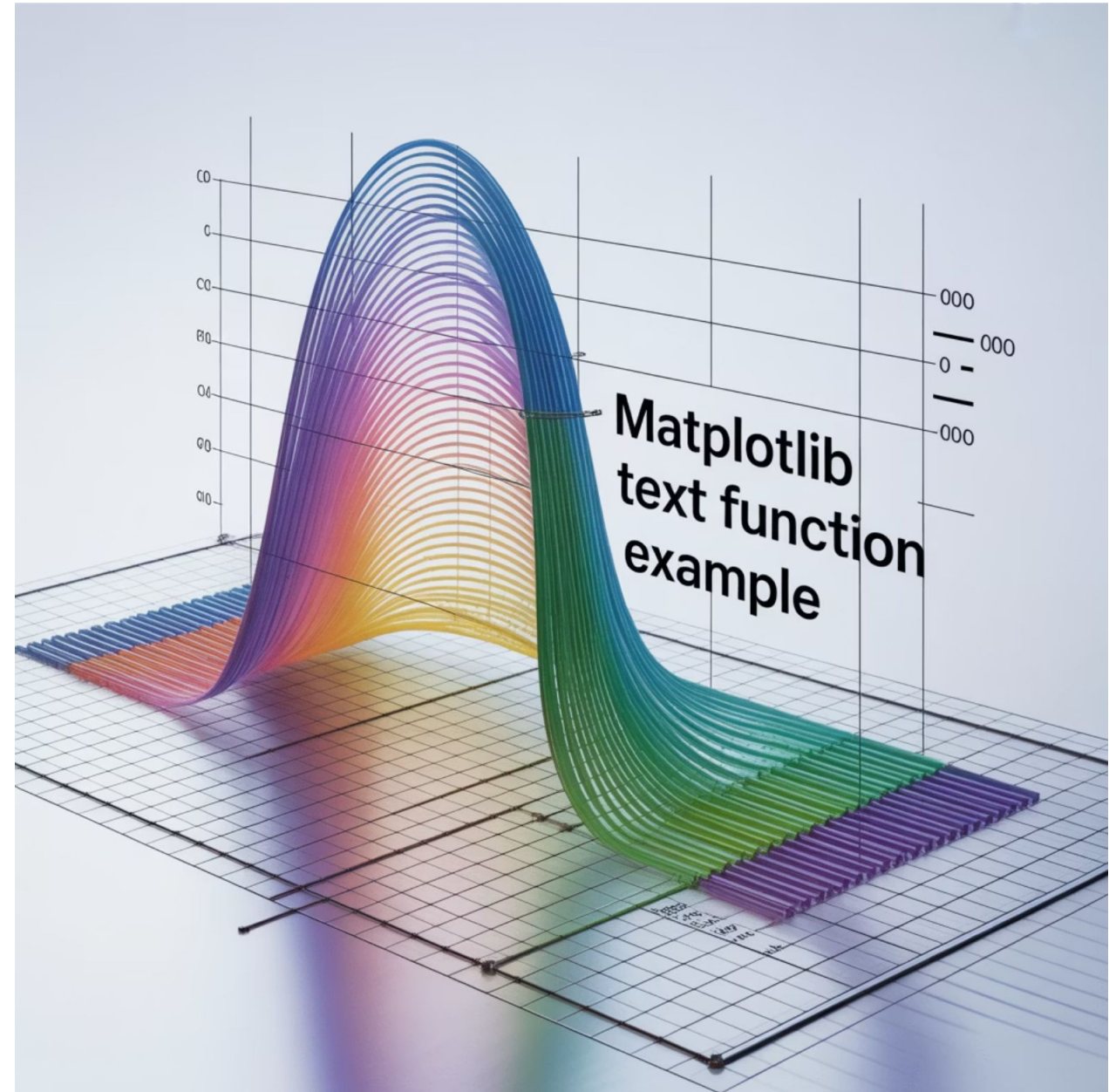


Agregar texto libre con text()

Cuando queremos colocar texto sin señalar un punto específico del gráfico, usamos text():

```
ax.text(2, 22, 'Tendencia creciente',  
        fontsize=10, color='green')
```

Esto es útil para observaciones generales o comentarios estratégicos dentro de la gráfica.

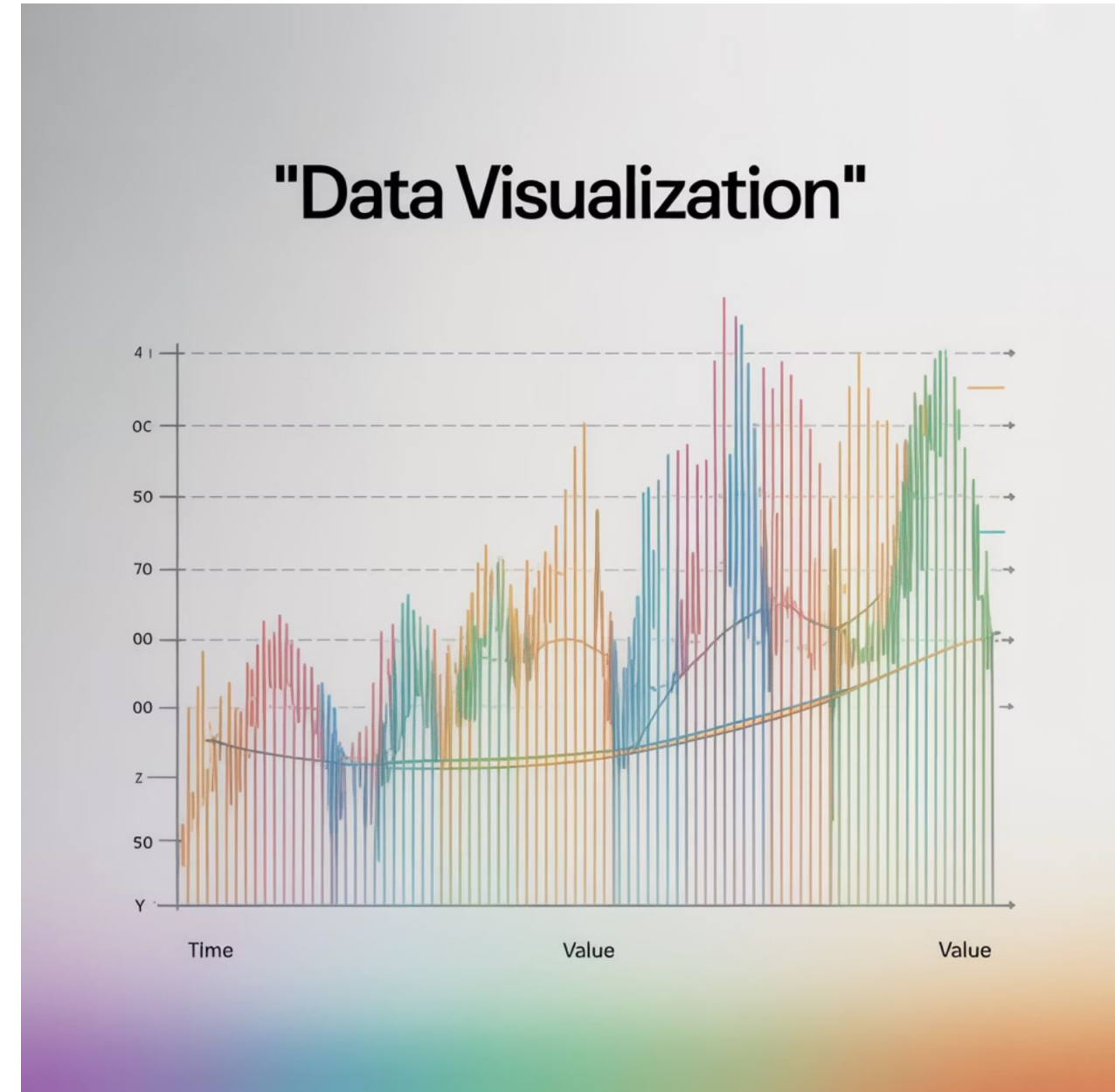


Dibujar líneas de referencia

- `ax.axhline(y=valor)` para líneas horizontales
- `ax.axvline(x=valor)` para líneas verticales

Estas líneas ayudan a establecer umbrales, promedios o puntos de corte.

```
ax.axhline(y=20, color='gray', linestyle='--')  
ax.axvline(x=2, color='blue', linestyle=':')
```

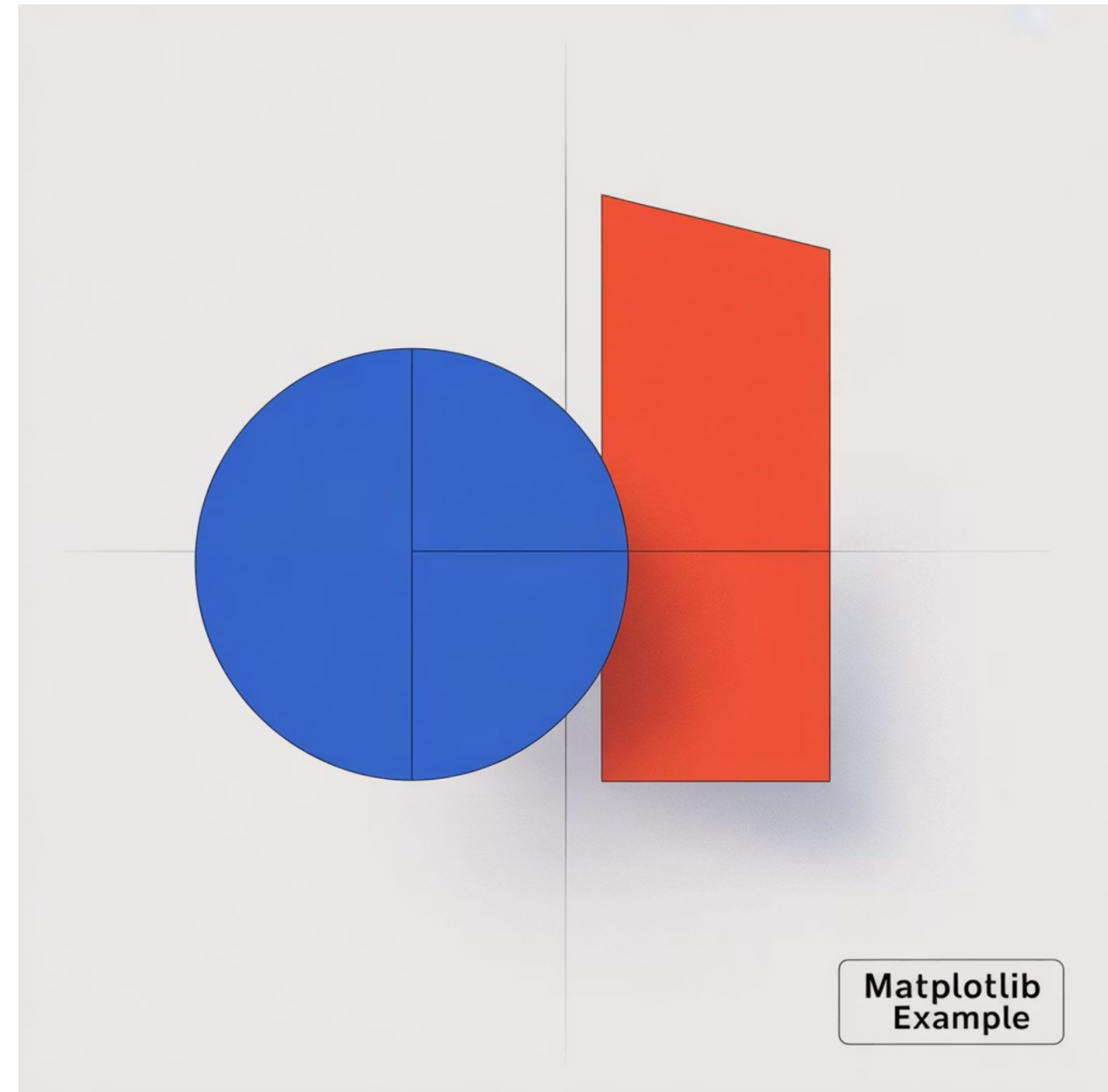


Elementos gráficos adicionales con patches

Para insertar formas (rectángulos, círculos, flechas), se usa el módulo `matplotlib.patches`:

```
import matplotlib.patches as patches
circle = patches.Circle((2, 20), radius=0.3,
color='orange', alpha=0.5)
ax.add_patch(circle)
```

Esto es útil para destacar zonas o crear visualizaciones con énfasis gráfico.



Buenas prácticas para anotaciones

Las anotaciones deben complementar el análisis, no saturar la visualización. Algunas recomendaciones:

Relevancia

Usar anotaciones para insights relevantes, no para repetir lo evidente.

Posicionamiento

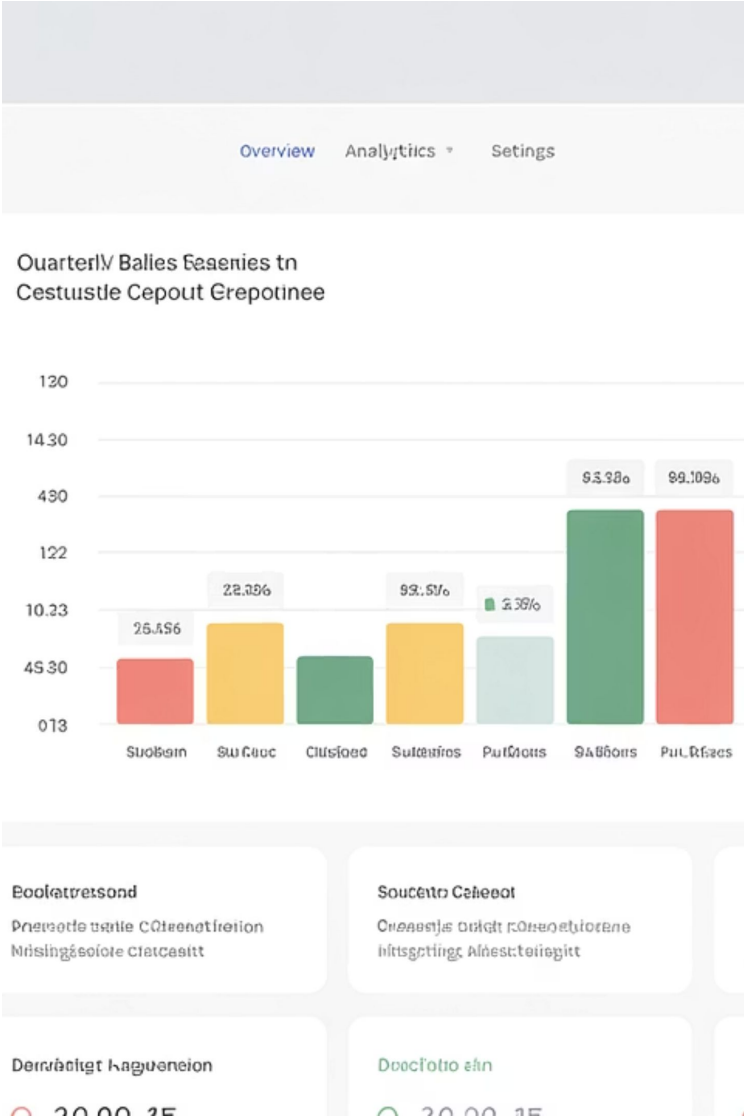
Elegir posiciones y flechas que no oculten los datos importantes.

Transparencia

Utilizar elementos gráficos con transparencia (alpha) para no distraer.

Valor analítico

Incluir explicaciones que aporten valor analítico o comunicacional.



Errores comunes y cómo evitarlos

Error	Causa probable	Solución recomendada
Texto se sobrepone a los datos	Ubicación automática inadecuada	Ajustar xytext o usar bbox para el fondo del texto
Exceso de anotaciones	Repetición o comentarios triviales	Priorizar solo los puntos clave
Flechas poco visibles	Estilo o color por defecto	Personalizar arrowprops con estilos adecuados

Las anotaciones enriquecen la visualización cuando se aplican con intención. Usarlas con criterio permite transformar un gráfico descriptivo en una narrativa visual poderosa.

A continuación, aprenderemos a guardar nuestras visualizaciones en archivos para informes, presentaciones o integraciones con otros sistemas.

Guardando los gráficos en un archivo

¿Cómo asegurarnos de que nuestros gráficos puedan ser reutilizados en informes o presentaciones sin perder calidad o formato?

Una visualización no cumple su propósito si no puede ser compartida. En análisis de datos aplicado, es fundamental exportar gráficos para su integración en informes técnicos, presentaciones o dashboards. Matplotlib permite guardar visualizaciones en múltiples formatos como PNG, PDF, SVG o JPEG, lo que ofrece versatilidad según el medio de difusión.

Guardar correctamente implica más que ejecutar una función. Es necesario controlar la resolución, el tamaño físico, el espacio alrededor del gráfico y evitar recortes indeseados.

Guardar con savefig()

La función savefig() es la herramienta principal para exportar gráficos:

```
fig, ax = plt.subplots()
ax.plot([1, 2, 3], [4, 5, 6])
plt.savefig('grafico.png')
```

Esto crea un archivo llamado grafico.png en el directorio actual.



Formatos disponibles



PNG

Imágenes para presentaciones o
páginas web



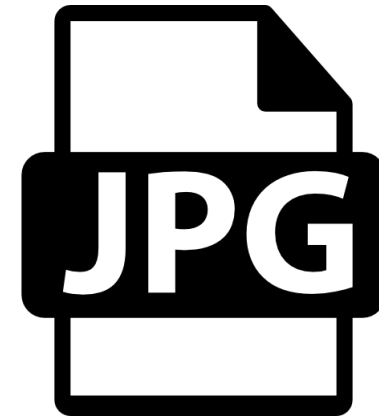
PDF

Documentos vectoriales de alta
resolución



SVG

Gráficos escalables para web o
impresión técnica



JPG

Fotografías o imágenes simples

Ajustar calidad y tamaño del archivo

Podemos controlar el tamaño del archivo final y su calidad mediante argumentos opcionales:

```
plt.savefig('grafico_hd.png', dpi=300, bbox_inches='tight')
```

- dpi=300: alta resolución (ideal para impresión)
- bbox_inches='tight': evita cortar etiquetas o márgenes

También se puede usar figsize al crear la figura para definir su tamaño físico:

```
fig, ax = plt.subplots(figsize=(8, 6))
```



Guardar sin mostrar el gráfico

En contextos automatizados o cuando no se requiere mostrar el gráfico en pantalla:

```
plt.savefig('reporte.png')  
plt.close()
```

Esto ahorra recursos, útil en scripts que generan múltiples gráficos.



Buenas prácticas para guardar gráficos

-  **Usar `bbox_inches='tight'`**
Siempre que se incluyan títulos o anotaciones para evitar recortes indeseados.
-  **Definir tamaño explícito**
Con `figsize()` según el medio donde se insertará el gráfico.
-  **Elegir formato según destino**
PDF para vector, PNG para web, según las necesidades específicas.
-  **Usar nombres descriptivos**
Evitar nombres genéricos de archivos, usar convenciones claras (ej. `ventas_Q1_2025.png`).

Errores comunes y cómo evitarlos

Error	Causa probable	Solución recomendada
Elementos cortados	Espacio insuficiente alrededor	Usar <code>bbox_inches='tight'</code>
Imagen borrosa	Resolución por defecto baja	Usar <code>dpi=300</code> en <code>savefig()</code>
Archivos sobrescritos	Nombres duplicados o poco claros	Agregar etiquetas únicas en el nombre

Saber exportar correctamente nuestros gráficos garantiza que la visualización conserve su claridad y profesionalismo en distintos entornos.

Con esto, ya estamos listos para explorar los distintos tipos de gráficos disponibles en Matplotlib y elegir cuál se adapta mejor a cada tipo de dato.

/* Tipos de gráficos en Matplotlib */



Tipos de gráficos en Matplotlib

Los distintos tipos de gráficos cumplen funciones específicas en el análisis exploratorio de datos. Elegir correctamente el tipo de visualización permite comunicar hallazgos de forma precisa y contextualizada según el tipo de variable (cuantitativa, categórica, continua o discreta) y el objetivo del análisis (comparar, distribuir, correlacionar, componer).

A continuación, revisaremos los principales tipos de gráficos en Matplotlib, con ejemplos aplicados y recomendaciones para su uso.

Diagrama de línea

El uso principal del diagrama de línea es mostrar evolución o tendencia de una variable continua a lo largo del tiempo.

Ejemplo

```
import matplotlib.pyplot as plt
meses = ['Ene', 'Feb', 'Mar', 'Abr', 'May']
ventas = [150, 200, 220, 180, 260]
plt.plot(meses, ventas, marker='o',
linestyle='-', color='blue')
plt.title('Ventas mensuales')
plt.xlabel('Mes')
plt.ylabel('Ventas en miles')
plt.grid(True)
plt.show()
```



Características clave del diagrama de línea

Características clave

- Usa plot() con listas ordenadas (tiempo, secuencia, etc.)
- Puede personalizarse con marker, linestyle y color
- Permite múltiples series sobre el mismo gráfico

Buenas prácticas

- Ordenar correctamente el eje X (temporal o secuencial)
- No saturar con más de 3–4 series simultáneas sin necesidad
- Usar marcadores (marker) para énfasis si hay pocos puntos



Errores comunes en diagramas de línea

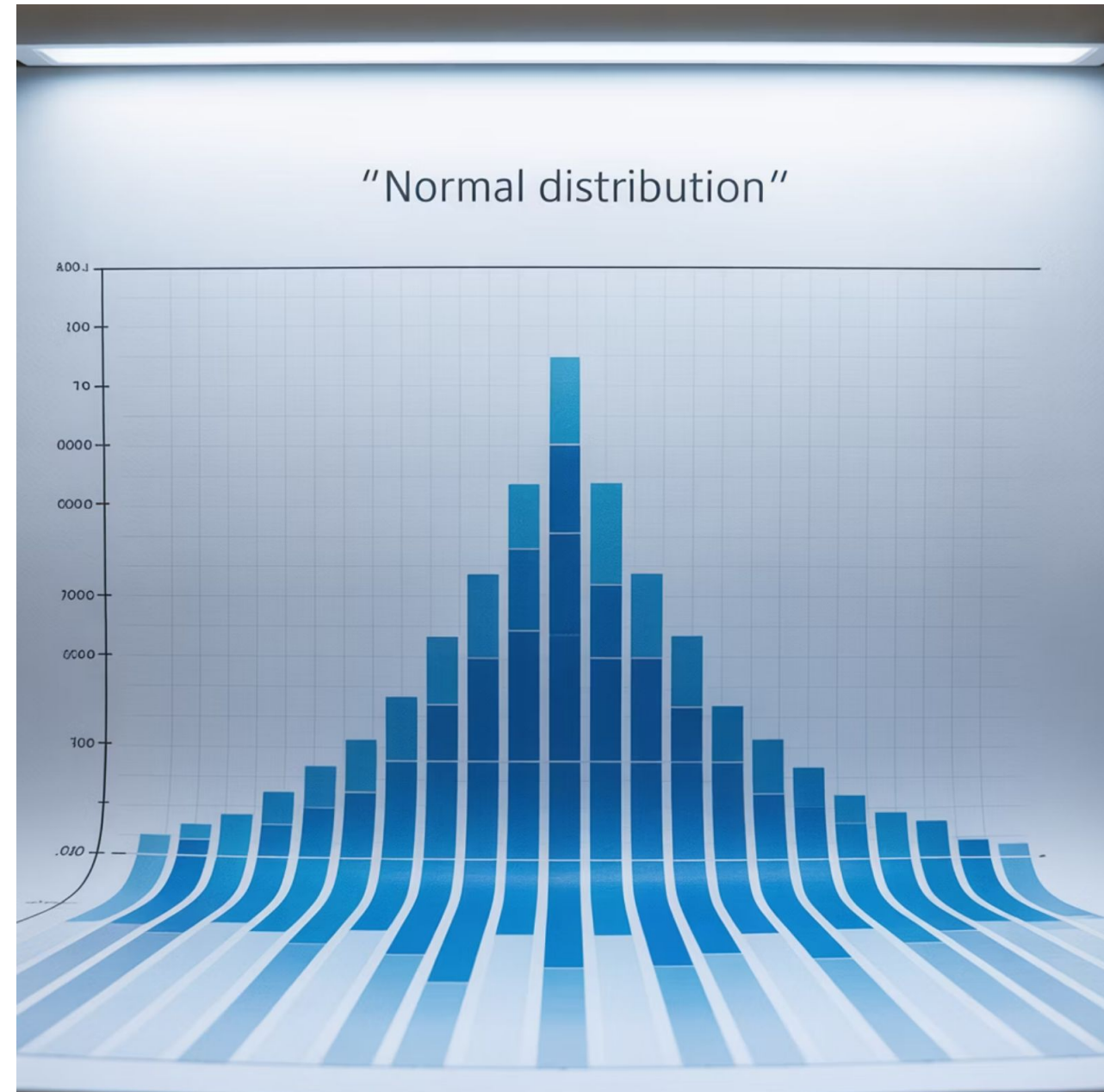
Error	Causa probable	Solución recomendada
Conexiones entre puntos erróneas	Datos desordenados	Ordenar datos según secuencia temporal
Dificultad de lectura	Colores similares entre series	Personalizar color y añadir leyenda

Histograma

Su uso principal es representar la distribución de frecuencias de una variable numérica continua. Es útil para analizar la forma de la distribución (simétrica, sesgada, multimodal, etc.).

Ejemplo

```
import matplotlib.pyplot as plt
import numpy as np
data = np.random.normal(loc=50, scale=10,
size=200)
plt.hist(data, bins=10, color='skyblue',
edgecolor='black')
plt.title('Distribución de puntajes')
plt.xlabel('Puntaje')
plt.ylabel('Frecuencia')
plt.grid(True)
plt.show()
```



Características clave del histograma

Características clave

- Usa hist() y define número de bins para agrupar datos
- Ideal para examinar la forma y dispersión de los datos

Buenas prácticas

- Usar entre 10 y 20 bins para distribuciones generales
- Aplicar edgecolor para mejorar legibilidad
- Normalizar con density=True si se desea representar proporciones



Errores comunes en histogramas

Error	Causa probable	Solución recomendada
Forma engañosa del histograma	Número de bins inadecuado	Ajustar el número de bins
Frecuencias poco claras	Sin bordes definidos	Usar edgecolor='black'

Boxplot

Este tipo de diagrama se utiliza principalmente para visualizar la dispersión y presencia de outliers en una o más series numéricas.

Ejemplo

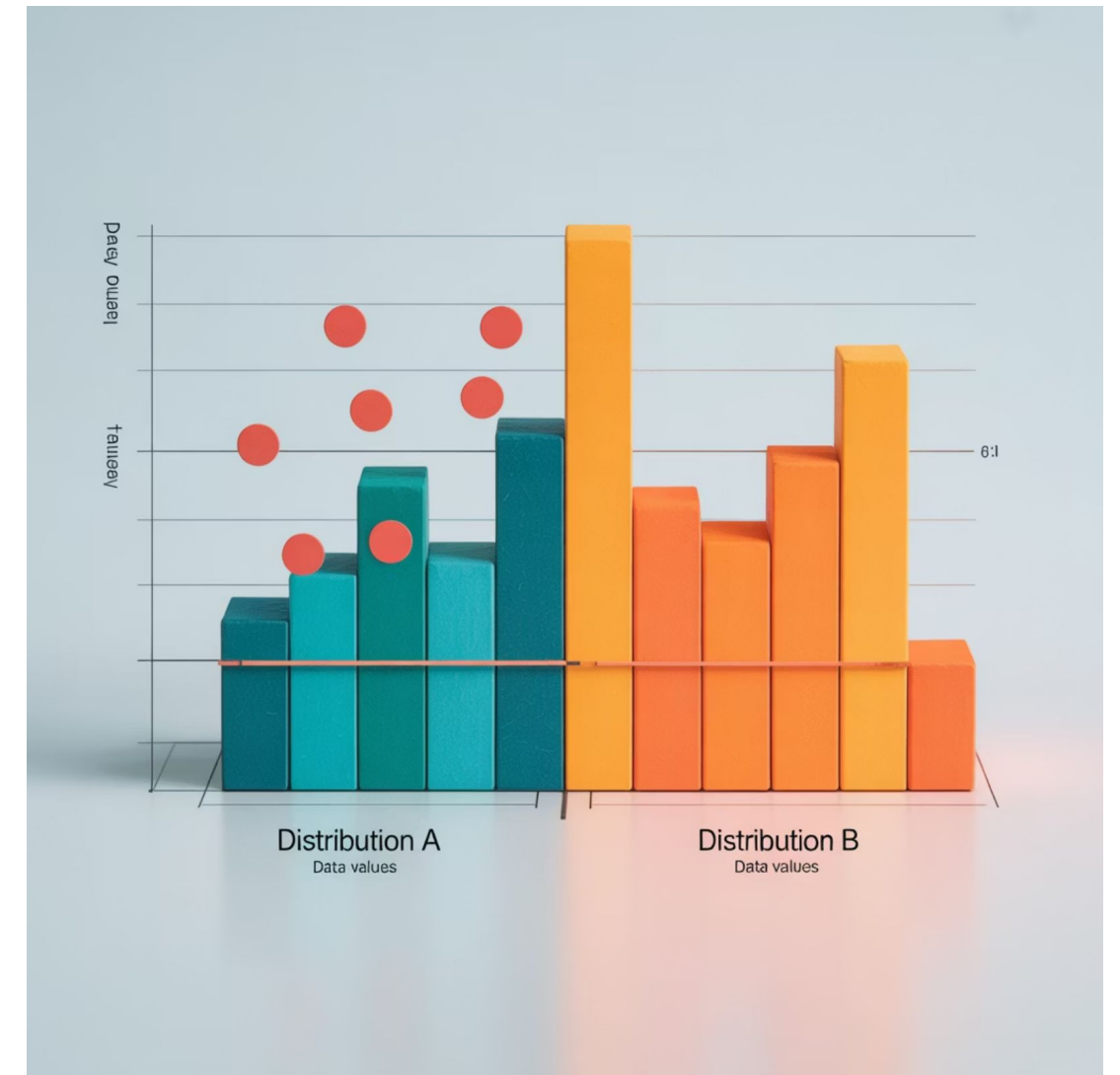
```
import matplotlib.pyplot as plt
import numpy as np
data = [np.random.normal(100, 10, 200), np.random.normal(80, 15, 200)]
plt.boxplot(data, labels=['Grupo A', 'Grupo B'])
plt.title('Comparación de distribuciones')
plt.ylabel('Valor')
plt.grid(True)
plt.show()
```

Características clave

- Usa boxplot() para representar estadísticos resumidos (Q1, Q2, Q3, IQR)
- Muestra valores atípicos como puntos aislados

Buenas prácticas

- Comparar múltiples grupos en una sola figura
- Etiquetar bien los grupos con labels=[]
- Combinar con histogramas si se desea ver forma y dispersión



Errores comunes en boxplots

Error	Causa probable	Solución recomendada
No se identifican bien los outliers	Rango vertical muy amplio	Ajustar límites del eje Y
Confusión en grupos	Falta de etiquetas	Usar labels y colores diferenciados

Diagrama de dispersión (Scatter Plot)

El principal uso de este tipo de diagramas es el de visualizar relaciones, correlaciones o patrones entre dos variables numéricas.

Ejemplo

```
import matplotlib.pyplot as plt
import numpy as np
x = np.random.rand(100)
y = 2 * x + np.random.normal(0, 0.1, 100)
plt.scatter(x, y, color='purple', alpha=0.6)
plt.title('Relación entre variables')
plt.xlabel('X')
plt.ylabel('Y')
plt.grid(True)
plt.show()
```

Características clave

- Usa scatter() con dos listas o arrays
- Se puede modificar tamaño (s), color (c) y transparencia (alpha)



Buenas prácticas y errores comunes en diagramas de dispersión

Buenas prácticas

- Usar alpha para mejorar visualización con muchos puntos
- Añadir líneas de tendencia si se desea reforzar interpretación

Errores comunes

Error	Causa probable	Solución recomendada
Puntos superpuestos	Muchos datos similares	Usar $\alpha < 1$ para transparencia
Difícil distinguir grupos	Color único para todas las clases	Usar color por categoría



Diagrama de barras

Se utiliza principalmente para comparar cantidades o frecuencias entre categorías nominales o discretas.

Ejemplo

```
categorias = ['A', 'B', 'C', 'D']
valores = [23, 17, 35, 29]
plt.bar(categorias, valores, color='teal')
plt.title('Distribución por categoría')
plt.xlabel('Categoría')
plt.ylabel('Cantidad')
plt.grid(True, axis='y')
plt.show()
```

Características clave

- Usa `bar()` para gráficos verticales
- Usa `barh()` para horizontales (ideal si las etiquetas son largas)



Buenas prácticas y errores comunes en diagramas de barras

Buenas prácticas

- Ordenar las barras de mayor a menor si no hay orden natural
- Usar colores para destacar diferencias sin exagerar

Errores comunes

Error	Causa probable	Solución recomendada
Dificultad para leer etiquetas	Etiquetas largas o densas	Usar <code>barh()</code> o rotar etiquetas
Barras poco distinguibles	Color uniforme o poco contraste	Personalizar colores y bordes



Diagrama de torta (Pie Chart)

Este tipo de diagrama se utiliza principalmente para representar proporciones relativas en relación con un total.

Ejemplo

```
etiquetas = ['Producto A', 'Producto B',  
'Producto C']  
porcentajes = [45, 30, 25]  
plt.pie(porcentajes, labels=etiquetas,  
autopct='%1.1f%%', startangle=90)  
plt.title('Participación de mercado')  
plt.axis('equal')  
plt.show()
```

Características clave

- Usa pie() con labels, autopct y startangle
- axis('equal') asegura que el círculo no se deforme



Buenas prácticas y errores comunes en diagramas de torta

Buenas prácticas

- Usar con máximo 5–6 categorías para mantener legibilidad
- Preferir otros tipos de gráficos si se requiere precisión comparativa

Errores comunes

Error	Causa probable	Solución recomendada
Difícil interpretar proporciones	Muchas categorías pequeñas	Consolidar en "Otros" si es necesario
Gráfico no circular	Falta de axis('equal')	Incluir <code>plt.axis('equal')</code>



Selección del tipo de gráfico adecuado

La correcta elección del tipo de gráfico es clave para comunicar datos de forma clara, efectiva y profesional. Cada visualización —líneas, histogramas, boxplots, dispersión, barras y tortas— cumple una función distinta según la naturaleza del dato y el objetivo del análisis.

Entender cuándo y cómo usarlos no solo mejora la calidad de los informes, sino también la capacidad de las personas analistas para interpretar, explicar y tomar decisiones basadas en evidencia visual sólida.

Actividad guiada: Creación de una colección de visualizaciones aplicadas



Ejercicio

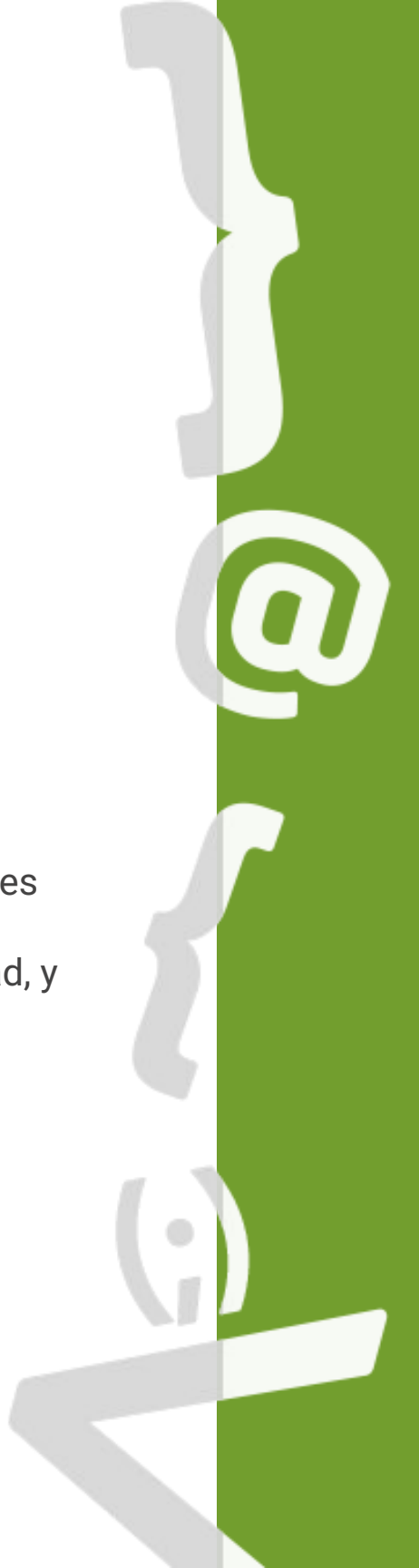
Explorando patrones de uso con visualizaciones en Seaborn

Objetivo de la actividad

Aplicar los conceptos aprendidos sobre límites de ejes, anotaciones, guardado de gráficos y tipos de visualización para construir una colección de gráficos efectivos a partir de un conjunto de datos reales.

Situación problema

Estás trabajando en el área de análisis de datos de una empresa minorista. Se te ha solicitado generar visualizaciones claras y exportables que representen: la evolución de ventas mensuales, la distribución de ingresos por cliente, la dispersión entre cantidad de compras y edad, y la proporción de tipos de producto más vendidos. El equipo requiere que los gráficos incluyan límites personalizados, anotaciones sobre valores extremos y una exportación adecuada para informes ejecutivos.



Ejercicio

Instrucciones

1

Cargar datos

Carga el archivo datos_clientes.csv.

2

Revisar columnas

Revisa las siguientes columnas: mes, ventas_mensuales, edad, compras_anuales, tipo_producto, ingreso_anual.

3

Crear gráficos

Crea los siguientes gráficos aplicando lo aprendido:

- Gráfico de línea: evolución de ventas_mensuales.
- Histograma: distribución de ingreso_anual.
- Boxplot: ingresos anuales por tipo_producto.
- Gráfico de dispersión: relación entre edad y compras_anuales.
- Gráfico de torta: participación de cada tipo_producto.

Ejercicio

Instrucciones

4

Añadir anotaciones

Añade al menos una anotación y define límites manuales en uno de los gráficos.

5

Exportar gráficos

Exporta al menos dos gráficos con calidad de impresión (dpi=300, bbox_inches='tight').

Código base sugerido

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.read_csv('datos_clientes.csv')
# Ejemplo de gráfico de dispersión con anotación y límites
plt.figure(figsize=(8, 5))
plt.scatter(df['edad'], df['compras_anuales'], alpha=0.6)
plt.xlabel('Edad')
plt.ylabel('Compras anuales')
plt.title('Relación entre edad y compras')
plt.xlim(18, 70)
plt.ylim(0, 100)
plt.annotate('Cliente destacado', xy=(45, 95), xytext=(50,
85), arrowprops=dict(arrowstyle='->', color='red'))
plt.grid(True)
plt.tight_layout()
plt.savefig('edad_vs_compras.png', dpi=300,
bbox_inches='tight')
plt.show()
```

Resumen

Límites de ejes

Aprendimos a fijar los límites de los ejes en un gráfico para controlar la percepción visual de los datos.

Anotaciones

Incorporamos anotaciones, flechas y dibujos para resaltar valores clave o eventos importantes en una visualización.

Exportación

Guardamos gráficos en diferentes formatos y calidades, con control de resolución y márgenes.

Tipos de gráficos

Exploramos los principales tipos de gráficos utilizados en análisis exploratorio: líneas, histogramas, boxplots, dispersión, barras y tortas.

Criterios de selección

Aplicamos criterios técnicos y comunicacionales para elegir el tipo de gráfico más adecuado según el tipo de dato y objetivo del análisis.



Próxima sesión...

Prueba.

{desafío}
latam_



Preguntas de cierre

¿Qué función de Matplotlib permite controlar el rango visualizado de un gráfico?

¿Cuáles son las diferencias entre `annotate()` y `text()` al agregar elementos en un gráfico?

¿Qué consideraciones debes tener al guardar un gráfico con alta calidad para impresión?

¿Qué tipo de gráfico utilizarías para mostrar la relación entre dos variables numéricas? ¿Y para comparar categorías?

¿Por qué no siempre es recomendable usar un gráfico de torta, incluso si queremos mostrar proporciones?